

✓ Preâmbulo

Imports, funções, downloads e instalação do Pytorch.

```
# Basic imports.
import os
import time
import numpy as np
import torch

import torch
from torch.autograd import Variable
import torch.nn.functional as F

from torch import nn
from torch import optim

from torch.utils.data import DataLoader
from torch.utils import data
from torch.backends import cudnn

from torchvision import models
from torchvision import datasets
from torchvision import transforms

from skimage import io

from sklearn import metrics

from matplotlib import pyplot as plt

%matplotlib inline

cudnn.benchmark = True
```

✓ Casting para o dispositivo correto

Como usaremos processamento vetorial principalmente em GPUs para aprendizado profundo, primeiramente é possível verificar se há uma GPU disponível com o trecho de código abaixo, armazenando os tensores nos dispositivos apropriados.

```
# Checking if GPU/CUDA is available.
if torch.cuda.is_available():
    device = torch.device('cuda')
else:
    device = torch.device('cpu')

print(device)

cuda
```

✓ Intro MLP

O perceptron e a camada `nn.Linear`

A camada Linear do Pytorch ([nn.Linear](#)) é responsável por aplicar uma transformação linear no dado de entrada. Esta camada recebe como parâmetro a dimensão (número de *features*) da entrada e da saída (que na verdade, representa o número de neurônios dessa camada). Por padrão o bias já é incluído. Um perceptron pode ser facilmente representado como a seguir, desconsiderando a função de ativação:

```
perceptron = nn.Linear(in_dimension, 1)
```

Mas de uma forma geral, uma camada Linear com diversas *features* de entrada e diversas *features* de saída pode ser representada como:

```
nn.Linear(in_features, out_features)
```

```
linear = nn.Linear(2,3)
print(linear)
```

```
Linear(in_features=2, out_features=3, bias=True)
```

Como é possível ver no código abaixo, o Pytorch já inicia os pesos da camada aleatoriamente.

```
for p in linear.parameters():
    print(p)
```

```
Parameter containing:
tensor([[ 0.3552, -0.6454],
        [ 0.2075,  0.0082],
        [-0.6106,  0.1468]], requires_grad=True)
Parameter containing:
tensor([ 0.4343, -0.1505, -0.6872], requires_grad=True)
```

O **forward** consiste em passar seu dado de entrada pela rede, gerando um resultado ao final. Considerando a camada linear instanciada anteriormente, o resultado do forward é o mesmo do somatório da multiplicação de seus pesos pelas respectivas entradas juntamente com o bias:

$$w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

No Pytorch, realizamos o **forward** chamando a função onde nossa rede/modelo está instanciada, conforme exemplo abaixo.

```
perceptron = nn.Linear(2,1)
X = torch.FloatTensor([2,3]) # dado de entrada de exemplo considerando o perceptron definido como nn.Linear(2,1)
print('Pytorch: ', perceptron(X))
```

```
# acessamos os pesos do modelo com .weight e o bias com .bias
print('Manual: ', torch.mul(X, perceptron.weight).sum() + perceptron.bias)
```

```
Pytorch:  tensor([0.6101], grad_fn=<ViewBackward0>)
Manual:  tensor([0.6101], grad_fn=<AddBackward0>)
```

✓ Exemplo Perceptron simples

O código abaixo cria um perceptron simples usando `nn.Linear` e implemente o fluxo de treinamento para esse perceptron, ou seja, faz o forward nessa camada, calcula a loss, e otimiza a camada. Invista um pouco de tempo para entender a célula abaixo pois usaremos essa ideia para implementar a função de treino mais a frente.

```
def loss_fn(predict, label): # definindo a loss
    return torch.pow(label - predict, 2)

perceptron = nn.Linear(1,1) # Camada linear com 1 feature de entrada (mais o bias) e uma de saída
perceptron.to(device) # casting do perceptron para GPU
learning_rate = 0.01
print('Parametros iniciais: ', list(perceptron.parameters()))

dataset = [] # dados de exemplo que seguem a função y = 2x+3
for x in range(10):
    dataset.append((x, 2*x+3))

for epoch in range(101):
    epoch_loss = 0
    for iteration, data in enumerate(dataset):
        X, y = data
        X, y = torch.FloatTensor([X]).to(device), torch.FloatTensor([y]).to(device) # conversão para Tensor

        y_pred = perceptron(X) # forward
        loss = loss_fn(y_pred, y) # calcula a loss

        epoch_loss += loss.item()
        loss.backward()
        with torch.no_grad():
            for param in perceptron.parameters():
                param -= learning_rate * param.grad # atualização dos parametros (pesos e bias) com base no gradiente
                param.grad.zero_() # resetando o gradiente

    if epoch % 10 == 0:
        print("Epoch {} - loss: {}".format(epoch, epoch_loss))
print('Parametros finais: ', list(perceptron.parameters()))
```

```
Parametros iniciais: [Parameter containing:
tensor([[-0.8763]], device='cuda:0', requires_grad=True), Parameter containing:
tensor([0.1583], device='cuda:0', requires_grad=True)]
Epoch 0 - loss: 369.5941658001393
```

```
Epoch 10 - loss: 3.658416314981878
Epoch 20 - loss: 1.26431422540918
Epoch 30 - loss: 0.43693560117390007
Epoch 40 - loss: 0.15100097516551614
Epoch 50 - loss: 0.05218461245385697
Epoch 60 - loss: 0.018034642929706024
Epoch 70 - loss: 0.006232386516785482
Epoch 80 - loss: 0.0021539658882829826
Epoch 90 - loss: 0.0007444477259923588
Epoch 100 - loss: 0.0002572918456280604
Parametros finais: [Parameter containing:
tensor([[2.0010]], device='cuda:0', requires_grad=True), Parameter containing:
tensor([2.9907], device='cuda:0', requires_grad=True)]
```

```
print(perceptron(torch.FloatTensor([20]).to(device))) # forward do valor 20 para conferir resultado, saida deve ser aproxim
tensor([43.0103], device='cuda:0', grad_fn=<ViewBackward0>)
```

> O módulo `nn.Sequential`

Na prática, criaremos redes com diversas camadas. O bloco `nn.Sequential` permite agrupar as camadas de forma sequencial para que o forward seja realizado na ordem desejada. Veja um exemplo para um *Multilayer Perceptron (MLP)* abaixo.

↳ 2 células ocultas

✓ Sua vez

Vamos agora treinar um MLP simples em dados aleatórios. A célula abaixo define as dimensões de entrada e saída e gera os dados aleatórios.

```
# N is batch size; D_in is input dimension;
# H is hidden dimension; D_out is output dimension.
N, D_in, H, D_out = 64, 1000, 100, 10

# Create random Tensors to hold inputs and outputs
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

# Casting tensors to the appropriate device.
x = x.to(device)
y = y.to(device)

# Printing sizes of tensors.
print('x: ', x.size())
print('y: ', y.size())

x: torch.Size([64, 1000])
y: torch.Size([64, 10])
```

Modifique o código abaixo para criar um módulo `nn.Sequential` de nome **model** que representa um MLP com, pelo menos, uma camada escondida (seguindo os valores N, D_in, H e D_out definidos anteriormente), **usando um ReLU como função de ativação entre as camadas**.

```
# Use the nn package to define our model.
model = nn.Sequential(
    nn.Linear(D_in, H),
    nn.ReLU(),
    nn.Linear(H, D_out)
)

model.to(device) # sempre eh necessario fazer o casting da rede para joga-la para GPU
print(model)

Sequential(
  (0): Linear(in_features=1000, out_features=100, bias=True)
  (1): ReLU()
  (2): Linear(in_features=100, out_features=10, bias=True)
)
```

Abaixo, definimos uma loss e um otimizador usando o Torch. Não se preocupem como isso agora, pois iremos ver em detalhes como definir e usar diferentes losses and otimizadores com o Torch.

```
# Use the nn package to define our loss function.
loss_mse = nn.MSELoss(reduction='sum').to(device)
```

```
# Use the optim package to define an Optimizer that will update the weights of
# the model for us. Here we will use SGD; the optim package contains many other
# optimization algorithms. The first argument tells the
# optimizer which Tensors it should update.
learning_rate = 1e-4

optimizer = optim.SGD(model.parameters(), lr=learning_rate)
```

Implemente abaixo a forward e cálculo da loss como feito anteriormente. Estude essa função, pois usaremos esse fluxo de treino mais a frente.

```
# Creating list of losses for each epoch.
loss_list = []

# Iterating over epochs.
for epoch in range(500):

    #####
    # Implemente sua solução aqui
    #####

    # 1. Forward pass: passa os dados de entrada 'x' pelo modelo
    y_pred = model(x)

    # 2. Calcula a loss usando a função MSE instanciada anteriormente
    loss = loss_mse(y_pred, y)

    if (epoch + 1) % 10 == 0:
        print('Epoch ' + str(epoch + 1) + ': loss = ' + str(loss.item()))

    # Updating list of losses for printing.
    loss_list.append(loss.item())

    # Before the backward pass, use the optimizer object to zero all of the
    # gradients for the variables it will update (which are the learnable
    # weights of the model). This is because by default, gradients are
    # accumulated in buffers( i.e, not overwritten) whenever .backward()
    # is called. Checkout docs of torch.autograd.backward for more details.
    optimizer.zero_grad()

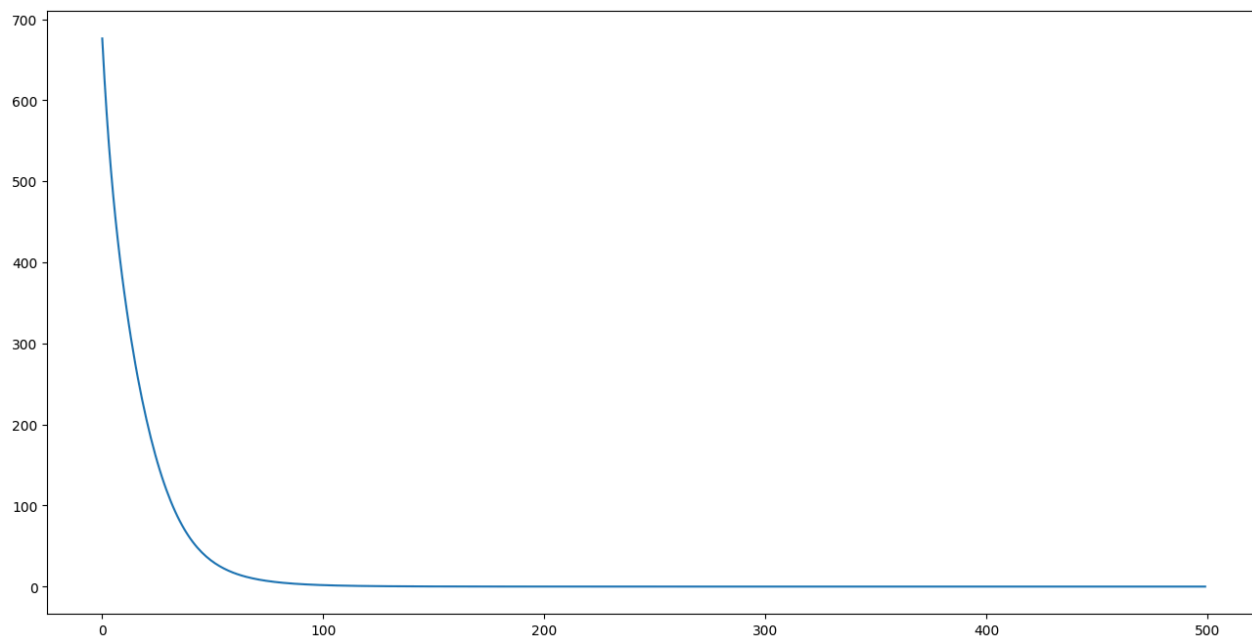
    # Backward pass: compute gradient of the loss with respect to model
    # parameters
    loss.backward()

    # Calling the step function on an Optimizer makes an update to its
    # parameters
    optimizer.step()
```

```
Epoch 10: loss = 382.77032470703125
Epoch 20: loss = 218.36715698242188
Epoch 30: loss = 119.4732666015625
Epoch 40: loss = 62.667572021484375
Epoch 50: loss = 32.71589660644531
Epoch 60: loss = 17.456565856933594
Epoch 70: loss = 9.601541519165039
Epoch 80: loss = 5.448415756225586
Epoch 90: loss = 3.175386428833008
Epoch 100: loss = 1.890944480895996
Epoch 110: loss = 1.150588870048523
Epoch 120: loss = 0.7113106846809387
Epoch 130: loss = 0.4468466341495514
Epoch 140: loss = 0.28485262393951416
Epoch 150: loss = 0.18399491906166077
Epoch 160: loss = 0.12026926130056381
Epoch 170: loss = 0.07944609224796295
Epoch 180: loss = 0.052986029535532
Epoch 190: loss = 0.03563859313726425
Epoch 200: loss = 0.02415299043059349
Epoch 210: loss = 0.016487881541252136
Epoch 220: loss = 0.011332989670336246
Epoch 230: loss = 0.00783782359212637
Epoch 240: loss = 0.005452428478747606
Epoch 250: loss = 0.0038134802598506212
Epoch 260: loss = 0.002680413657799363
Epoch 270: loss = 0.001892751082777977
Epoch 280: loss = 0.001341940718702972
Epoch 290: loss = 0.0009548659436404705
Epoch 300: loss = 0.0006820702692493796
Epoch 310: loss = 0.0004889472620561719
Epoch 320: loss = 0.00035164307337254286
Epoch 330: loss = 0.00025364436442032456
Epoch 340: loss = 0.00018346193246543407
Epoch 350: loss = 0.0001330322993453592
```

```
Epoch 360: loss = 9.669810242485255e-05  
Epoch 370: loss = 7.043822552077472e-05  
Epoch 380: loss = 5.141779547557235e-05  
Epoch 390: loss = 3.7604128010571e-05  
Epoch 400: loss = 2.7551066523301415e-05  
Epoch 410: loss = 2.0218802092131227e-05  
Epoch 420: loss = 1.4860194823995698e-05  
Epoch 430: loss = 1.0937495972029865e-05  
Epoch 440: loss = 8.061715561780147e-06  
Epoch 450: loss = 5.949447768216487e-06  
Epoch 460: loss = 4.395560608827509e-06  
Epoch 470: loss = 3.2512880352442153e-06  
Epoch 480: loss = 2.407319698249921e-06  
Epoch 490: loss = 1.7845391084847506e-06  
Epoch 500: loss = 1.3237391840448254e-06
```

```
fig, ax = plt.subplots(1, 1, figsize=(16, 8))  
  
ax.plot(np.asarray(loss_list))  
  
plt.show()
```



Informação sobre outras camadas lineares, como `nn.Bilinear` e `nn.Identity`, podem ser vistas na documentação:

<https://pytorch.org/docs/stable/nn.html#linear-layers>